Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Codage des entiers relatifs, algorithme du complément à deux, arithmétique binaire et détection du dépassement de capacité (Overflow)

Synthèse exécutive : Ce cours aborde la manière dont les ordinateurs codent et manipulent physiquement les entiers signés en mémoire binaire sur N bits. En l'absence de composant électronique capable de stocker directement un signe '+' ou '-', l'architecture informatique s'appuie sur la convention du complément à deux. Ce système élimine le problème du double zéro (présent dans le codage Signe et Magnitude) et unifie l'addition et la soustraction au sein du même circuit additionneur de l'Unité Arithmétique et Logique (ALU). Le cours étudie la méthode de conversion en 3 étapes, les plages de valeurs dissymétriques, l'interprétation du bit de poids fort (MSB), la gestion de la retenue sortante (Carry out) et la règle stricte de détection du dépassement de capacité (Overflow).

Problématique matérielle et justification du Complément à Deux

Au niveau physique le plus bas de l'architecture d'un ordinateur, l'information numérique est conservée au sein de transistors et de bascules électroniques. Ces composants bistables ne reconnaissent que deux états de tension : 0 (absence de tension, 0V) et 1 (présence de tension, Vcc). Il est physiquement impossible de fabriquer une bascule stockant un signe '+' ou '-'. Pour représenter des entiers relatifs, les ingénieurs ont d'abord pensé au codage 'Signe + Magnitude' : le bit de poids fort (MSB) sert de signe (0 pour positif, 1 pour négatif) et les N-1 bits restants indiquent la valeur absolue. Cette approche possède deux graves lacunes : elle génère un double zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison et complexifie la logique de comparaison, et elle oblige à doubler la surface matérielle de l'Unité Arithmétique et Logique (ALU) en intégrant deux circuits séparés pour l'addition et la soustraction. Le complément à deux s'impose comme la solution universelle : le zéro devient unique (0000...0) et la soustraction A - B est convertie en une addition A + (-B), réutilisant directement le circuit additionneur binaire basique.

- Absence de signe physique dans les composants électroniques (transistors, bascules bistables ne stockant que 0V ou Vcc).
- Limites majeures du codage Signe et Magnitude (existence de +0 et -0, besoin de circuits matériels distincts pour addition et soustraction).
- Unification de l'ALU via le complément à deux (représentation unique du zéro et réalisation des soustractions A - B sous forme d'additions A + (-B)).

Cas concret enseignant : *La métaphore du compteur kilométrique analogique à 4 chiffres d'une voiture ancienne : bloqué à 0000 km, si l'on recule d'un kilomètre en marche arrière, le compteur bascule à 9999. En binaire sur 8 bits, faire 00000000 - 1 produit 11111111_2, ce qui définit formellement -1 en complément à deux.*

Algorithme de conversion et plage de valeurs sur N bits

Pour représenter un entier négatif -X en complément à deux sur N bits, une procédure exacte en trois étapes doit être exécutée. Première étape : convertir la valeur absolue X en binaire pur et la cadrer rigoureusement sur N bits en insérant tous les 0 nécessaires sur la gauche. Deuxième étape : inverser chaque bit de la chaîne (permuter les 0 en 1 et les 1 en 0), ce qui produit le complément à un. Troisième étape : ajouter la valeur 1 au nombre binaire obtenu pour former le complément à deux. Le bit le plus à gauche (MSB) de la représentation finale vaut 1 si le nombre est négatif, et 0 s'il est positif ou nul. La plage de valeurs représentables sur N bits est dissymétrique : elle s'étend de -2^(N-1) jusqu'à +(2^(N-1) - 1). Par exemple sur 8 bits, elle s'étend de -128 à +127. Cette dissymétrie vient du fait que le zéro (00000000) possède un MSB de 0 et est rattaché au domaine positif.

- Algorithme des 3 étapes : Valeur absolue sur N bits -> Complément à 1 (inversion) -> Complément à 2 (+1).

- Rôle du bit de poids fort (MSB) comme indicateur de signe (0 = positif, 1 = négatif).
- Dissymétrie de la plage de valeurs : [-2^(N-1), +(2^(N-1) - 1)].

Cas concret enseignant : *Piège d'évaluation classique répété par le professeur : il est ABSOLUMENT OBLIGATOIRE de cadrer la valeur absolue sur N bits (ex: 8 bits) AVANT d'inverser les bits. Si l'on inverse '1100' (+12 sur 4 bits) sans rajouter les 4 zéros de gauche '00001100', on obtient '0011' au lieu de '11110011', ce qui conduit à un résultat totalement faux en examen !*

Arithmétique binaire, Retenue (Carry Out) et Détection du Dépassement (Overflow)

Au sein d'un processeur, les registres possèdent une largeur matérielle fixe (8, 16, 32 ou 64 bits). Lorsqu'une addition binaire est effectuée en complément à deux, une retenue sortante (Carry Out) peut émerger au-delà du bit de poids fort (MSB). En arithmétique signée modulo 2^N, cette retenue sortante est tout simplement ignorée par l'ALU. Cependant, il ne faut pas confondre le Carry Out avec l'Overflow (dépassement de capacité). L'Overflow indique que le résultat mathématique de l'opération sort du domaine représentable [-2^(N-1), +2^(N-1)-1]. La règle formelle de détection d'un Overflow lors d'une addition de deux entiers signés est stricte : un Overflow survient si et seulement si on additionne deux nombres de même signe (deux positifs ou deux négatifs) et que le résultat présente un signe inverse (MSB altéré). Si l'on additionne un nombre positif et un nombre négatif, l'Overflow est physiquement et mathématiquement impossible.

- Calcul modulo 2^N et élimination de la retenue sortante (Carry Out).
- Définition du drapeau d'Overflow (OV) comme erreur de capacité arithmétique.
- Règle absolue d'Overflow : présence d'un dépassement si et seulement si l'addition de deux opérandes de même signe donne un résultat de signe opposé.

Cas concret enseignant : *L'anecdote du bogue d'Ariane 5 (vol 501, 1996) : une conversion non sécurisée d'un flottant 64 bits vers un entier signé 16 bits a causé un Overflow non capturé. L'Unité de Référence Inertielle a renvoyé une valeur corrompue, interprétée à tort comme une déviation de trajectoire par le calculateur, provoquant le braquage brutal des tuyères puis la destruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : Parce que le processeur travaille en arithmétique modulaire modulo 2^N sur un registre de N bits. La retenue sortante correspond à l'ajout de 2^N, valeur qui déborde de la taille du mot mémoire. En complément à deux, rejeter ce bit d'ordre 2^N préserve la justesse du résultat modulo 2^N pour les nombres signés.

Question étudiant : Comment l'ordinateur fait-il la différence entre le nombre binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : Matériellement, les registres et le circuit d'addition traitent les bits exactement de la même manière sans connaître leur signification. C'est le contexte d'exécution du programme et les instructions machine utilisées (par exemple des instructions de comparaison signées vs non signées) qui déterminent si la combinaison 11111111 doit être interprétée comme 255 ou comme -1.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler le codage binaire des deux opérandes, poser et calculer l'addition binaire correspondante, fournir le résultat décimal et analyser le drapeau d'Overflow.

Résolution pas à pas :
1. Codage de +35 sur 8 bits : 35 = 32 + 2 + 1 = 00100011_2. 2. Codage de -75 en complément à deux sur 8 bits : - Valeur absolue (+75) sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion des bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2. 3. Pose de l'addition binaire (+35) + (-75) : 00100011

(+35) + 10110101 (-75) ---------- 11011000_2 4. Interprétation du résultat : Le bit MSB vaut 1, le nombre est donc négatif. Pour retrouver sa valeur absolue, on réapplique le complément à deux sur 11011000_2 : - Inversion : 00100111_2 - Ajout de 1 : 00101000_2 = 32 + 8 = 40 décimal. Le résultat binaire 11011000_2 vaut donc -40 en décimal. 5. Analyse de l'Overflow : Les deux opérandes (+35 et -75) sont de signes opposés (MSB de +35 = 0, MSB de -75 = 1). Il est mathématiquement impossible d'avoir un Overflow lors de l'addition de deux nombres de signes différents.

Résultat : Résultat binaire : 11011000_2 (-40 décimal). Drapeau d'Overflow OV = 0 (pas de dépassement de capacité). Le calcul 35 - 75 = -40 est exact.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage exacte des entiers signés représentables sur un mot de 16 bits en complément à deux ?

R : La plage s'étend de -2^(16-1) à +(2^(16-1) - 1), c'est-à-dire de -32 768 à +32 767.

Q : Comment détecte-t-on de façon certaine la présence d'un Overflow lors d'une addition de deux entiers signés ?

R : Un Overflow survient si et seulement si l'addition de deux opérandes de même signe (deux positifs ou deux négatifs) produit un résultat de signe opposé.

Q : Quel est le résultat de l'application de l'algorithme du complément à deux sur la valeur zéro (00000000_2) sur 8 bits ?

R : 00000000_2 inversé donne 11111111_2. En ajoutant 1, on obtient 100000000_2. Sur 8 bits, le bit de retenue de gauche est ignoré, ce qui redonne 00000000_2. Le zéro possède donc une représentation unique.